

Advanced Robotics

Homework 4

Yusif Razzaq
GitHub Repo ↗

November 2025

1 Introduction

This assignment investigates the use of various state estimation algorithms for maintaining a belief about the true state of the agent using dynamics and measurement models. Within this common framework, three common filters were implemented and assessed on their performance with respect to the true state via simulation. Specifically, the Bayes filter was applied in a discrete setting, whereas an Extended Kalman Filter (EKF) and a Particle Filter (PF) were applied to continuous state estimation problems. The simulation results show the ability of these filters to predict the true state and characterize their estimation uncertainties.

2 Bayes Filter

For this implementation, we consider a simple gridworld where the agent can take steps in any cardinal direction (or stay in place) within the bounds of the state space while respecting obstacles. The robot state was modeled as a 3D state vector $x_t = (i_t, j_t, \theta_t)$, where (i, j) correspond to the column and row position, and θ denotes the orientation of the robot.

A Bayes filter was used to maintain a belief over every cell in the gridworld and was recursively updated at each time step. By using the provided state transition and measurement models, the belief was updated using prediction and measurement update steps. To validate the accuracy of the filter, the true robot state was simulated according to random inputs. Then, noisy sensor measurements were passed to the filter, and the error probability distribution over the grid cells was monitored.

The final belief distribution can be seen in Figure 1 and the position error over time is shown in Figure 2. These results show that the Bayes Filter was able to converge onto the true state estimate after enough measurements were received to resolve any symmetries in the environment.

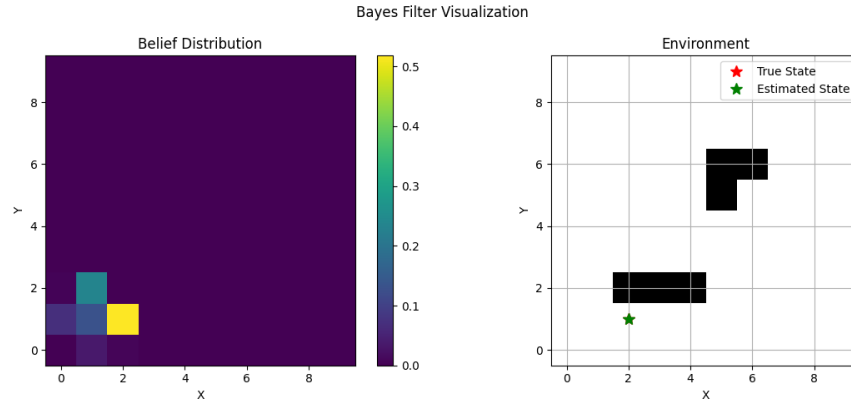


Figure 1: Final belief over robot state position using the Bayes Filter.

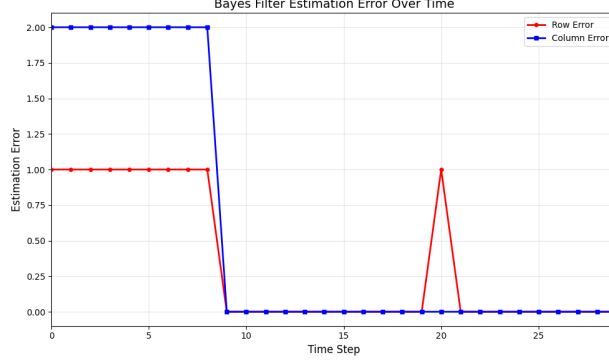


Figure 2: State estimation error over time using the Bayes Filter.

3 Extended Kalman Filter

Next, an Extended Kalman Filter (EKF) was implemented to track the position and orientation of an agent. The motion of the agent was modeled as a nonlinear unicycle according to the following dynamics equations:

$$x_{t+1} = x_t + v_{\text{eff}} \cos(\theta_t) \Delta t, \quad (1)$$

$$y_{t+1} = y_t + v_{\text{eff}} \sin(\theta_t) \Delta t, \quad (2)$$

$$\theta_{t+1} = \theta_t + \omega_{\text{eff}} \Delta t, \quad (3)$$

where:

$$v_{\text{eff}} = v \cdot s(x, y), \quad s(x, y) = 1 + 0.5 \sin(\omega_{\text{terr}} x) \cdot \cos(\omega_{\text{terr}} y),$$

and

$$\omega_{\text{eff}} = \frac{\omega}{|v_{\text{eff}}|} \cdot v_{\text{max}}.$$

3.1 Motion Jacobian Derivation

In order to perform the state prediction step in the EKF a Jacobian matrix F was derived as follows:

$$F = \frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial x_{t+1}}{\partial x_t} & \frac{\partial x_{t+1}}{\partial y_t} & \frac{\partial x_{t+1}}{\partial \theta_t} \\ \frac{\partial y_{t+1}}{\partial x_t} & \frac{\partial y_{t+1}}{\partial y_t} & \frac{\partial y_{t+1}}{\partial \theta_t} \\ \frac{\partial \theta_{t+1}}{\partial x_t} & \frac{\partial \theta_{t+1}}{\partial y_t} & \frac{\partial \theta_{t+1}}{\partial \theta_t} \end{bmatrix}.$$

Since $v_{\text{eff}} = v \cdot s(x, y)$, we first need to compute its spatial derivatives:

$$\frac{\partial v_{\text{eff}}}{\partial x} = v s_x, \quad \frac{\partial v_{\text{eff}}}{\partial y} = v s_y.$$

where,

$$s_x = \frac{\partial s}{\partial x} = 0.5 \omega_{\text{terr}} \cos(\omega_{\text{terr}} x) \cos(\omega_{\text{terr}} y)$$

$$s_y = \frac{\partial s}{\partial y} = -0.5 \omega_{\text{terr}} \sin(\omega_{\text{terr}} x) \sin(\omega_{\text{terr}} y)$$

Row 1: derivatives of x_{t+1} :

$$F_{1,1} = \frac{\partial}{\partial x_t}(x_t + v_{\text{eff}} \cos \theta_t \Delta t) = 1 + \Delta t (v s_x) \cos \theta_t,$$

$$F_{1,2} = \frac{\partial}{\partial y_t}(v_{\text{eff}} \cos \theta_t \Delta t) = \Delta t (v s_y) \cos \theta_t,$$

$$F_{1,3} = \frac{\partial}{\partial \theta_t}(v_{\text{eff}} \cos \theta_t \Delta t) = -\Delta t v_{\text{eff}} \sin \theta_t.$$

Row 2: derivatives of y_{t+1} :

$$F_{2,1} = \frac{\partial}{\partial x_t}(v_{\text{eff}} \sin \theta_t \Delta t) = \Delta t \sin \theta_t (v s_x),$$

$$F_{2,2} = \frac{\partial}{\partial y_t}(v_{\text{eff}} \sin \theta_t \Delta t) = \Delta t \sin \theta_t (v s_y),$$

$$F_{2,3} = \frac{\partial}{\partial \theta_t}(v_{\text{eff}} \sin \theta_t \Delta t) = \Delta t v_{\text{eff}} \cos \theta_t.$$

Row 3: derivatives of θ_{t+1} :

Note ω_{eff} is a function of x, y :

$$\omega_{\text{eff}}(x, y) = \frac{\omega v_{\text{max}}}{|v_{\text{eff}}|}$$

Differentiate:

$$\frac{\partial \omega_{\text{eff}}}{\partial x} = \omega v_{\text{max}} \frac{\partial}{\partial x}(|v_{\text{eff}}|^{-1}) = -\omega v_{\text{max}} \frac{v_{\text{eff}}}{|v_{\text{eff}}|^3} \frac{\partial v_{\text{eff}}}{\partial x}.$$

Substitute $\partial_x v_{\text{eff}} = v s_x$:

$$\frac{\partial \omega_{\text{eff}}}{\partial x} = -\omega v_{\text{max}} \frac{v_{\text{eff}} (v s_x)}{|v_{\text{eff}}|^3} = -\omega_{\text{eff}} \frac{v_{\text{eff}} (v s_x)}{|v_{\text{eff}}|^2},$$

Thus the Jacobian entries (multiply by Δt) are

$$F_{3,1} = -\Delta t \omega_{\text{eff}} \frac{v_{\text{eff}} (v s_x)}{|v_{\text{eff}}|^2}, \quad F_{3,2} = -\Delta t \omega_{\text{eff}} \frac{v_{\text{eff}} (v s_y)}{|v_{\text{eff}}|^2}, \quad F_{3,3} = 1.$$

3.2 Measurement Jacobian Derivation

The range-only measurement model is defined as follows:

$$z = h(\mathbf{x}) = \sqrt{(x - x_s)^2 + (y - y_s)^2}.$$

Then, the Jacobian which can be used in the measurement update step, $H = \frac{\partial h}{\partial \mathbf{x}}$, can be found according to the following derivation:

Define the differences in position:

$$\begin{aligned} d_x &= x - x_s, \\ d_y &= y - y_s, \\ r &= \sqrt{d_x^2 + d_y^2}. \end{aligned}$$

Using the chain rule, the partial derivatives of r with respect to the state variables are:

$$\frac{\partial r}{\partial x} = \frac{\partial r}{\partial d_x} \frac{\partial d_x}{\partial x} = \frac{d_x}{r}, \quad \frac{\partial r}{\partial y} = \frac{\partial r}{\partial d_y} \frac{\partial d_y}{\partial y} = \frac{d_y}{r}, \quad \frac{\partial r}{\partial \theta} = 0,$$

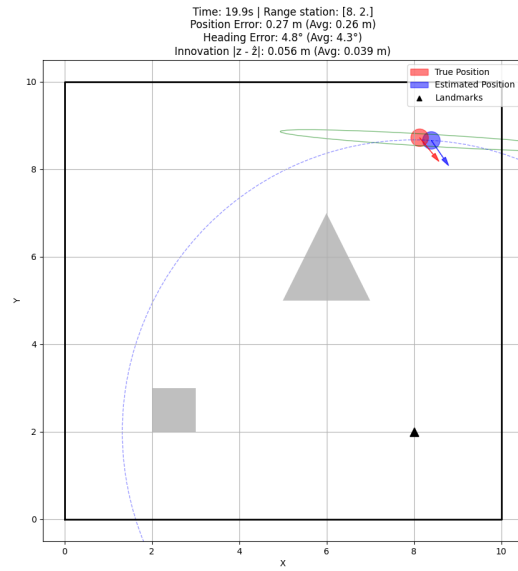


Figure 3: Final state estimate using the Extended Kalman Filter.

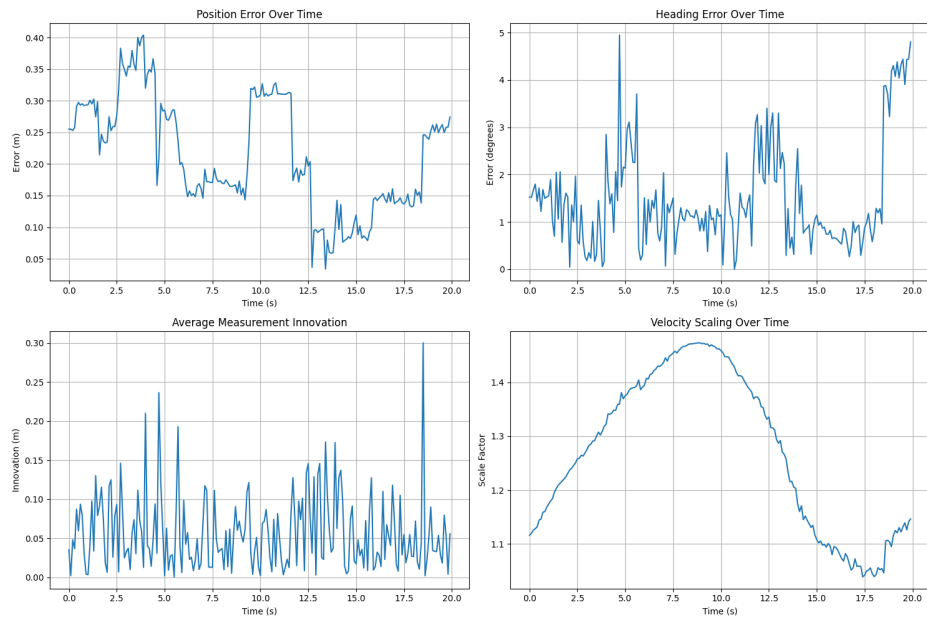


Figure 4: State estimation error over time using the Extended Kalman Filter.

Collecting these derivatives into a row vector gives the 1×3 Jacobian:

$$H = \begin{bmatrix} \frac{\partial r}{\partial x} & \frac{\partial r}{\partial y} & \frac{\partial r}{\partial \theta} \end{bmatrix} = \begin{bmatrix} \frac{d_x}{r} & \frac{d_y}{r} & 0 \end{bmatrix}.$$

To handle cases where the robot is very close to the station, (i.e., $r \rightarrow 0$), the Jacobian is set to zero to avoid undefined behavior:

$$H = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}, \quad \text{for } r < 1e-9.$$

Finally, a simulation was conducted to determine the performance of the EKF. The final state estimate can be seen in Figure 3 and the state error over time is shown in Figure 4. These results show that the EKF was able to track the true state estimate during the course of the maneuver with the positional and orientation errors remaining under $0.41m$ and 5° , respectively.

4 Particle Filter

The last filter implemented was the particle filter (PF) which is initialized by using N particles sampled uniformly over the state space. Each particle is propagated using a dynamics model, then the expected measurement at that state is calculated. Next, each particle is weighted in accordance with the likelihood that the expected measurement agrees with the actual measurement. Then, a set of N new particles are regenerated using a multinomial resampling strategy.

At each time step, we can return a state estimate using a weighted mean of the particles which was compared to the true state in simulation. The final state distribution of particles can be seen in Figure 5 and the state error over time is shown in Figure 6. This shows that all the particles eventually clustered around the true state as likelihood of receiving the actual measurements only agreed with one hypothesis set.

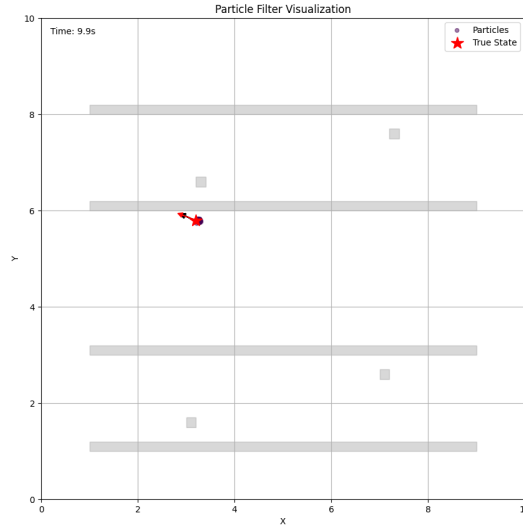


Figure 5: Final particles distribution of Particle Filter.

5 Conclusion

In this assignment, we implemented various state estimation techniques under different motion and measurement models. First, we applied the Bayes Filter to a Gridworld environment, and validated that the

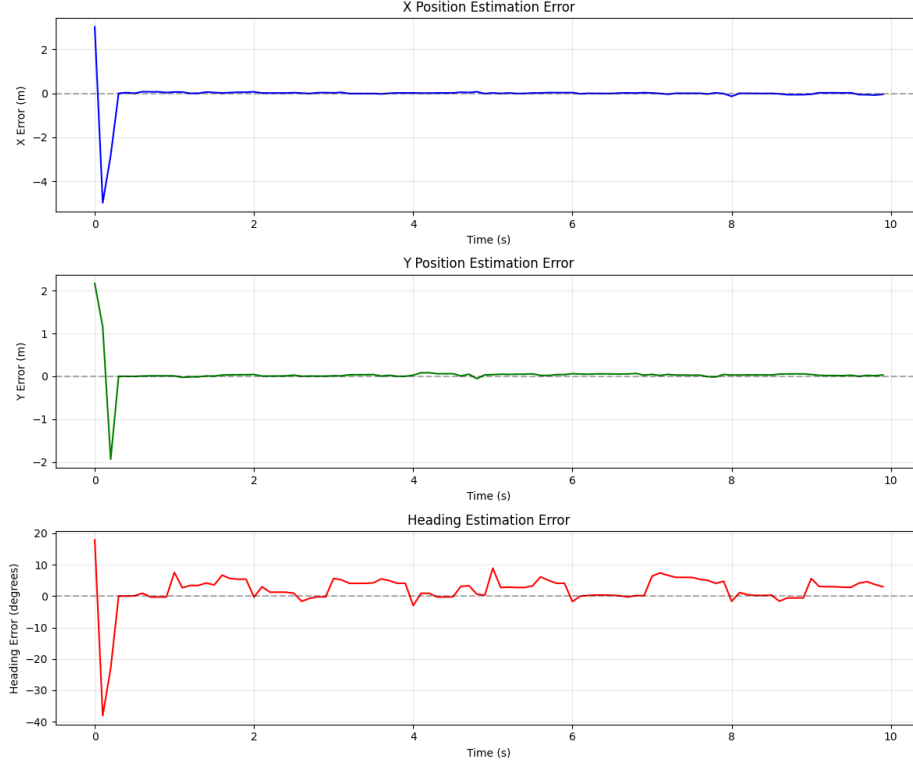


Figure 6: State estimation error over time using the Particle Filter.

probability density converges onto the true state after enough measurements agree. Next, we tested an EKF on a range-only measurement model and found that the estimation errors were reasonable despite the presence of noise in the motion and sensor models. Finally, we showed that the PF was able to resolve the many hypotheses it was initialized with, resulting in all the particles clustering around the true state.

All these methods proved successful on their respective problem setups. However, since the environments and models differed between filters, it is difficult to make direct comparisons between the analyzed approaches. The PF was the easiest to implement because no Jacobians needed to be calculated, and simple evaluation of the dynamics and measurement models was required. The Bayes Filter would in some instances take time to converge as the environment was prone to producing identical measurements even at differing states.¹

¹AI tools were used for the following tasks: Auto-complete in code editor, generating code for error plots (for BF & PF), correcting mistakes in F and H derivations, formatting derivations in LaTeX, correcting grammatical mistakes in the report.